

Ponteiros

melise@unifei.edu.br

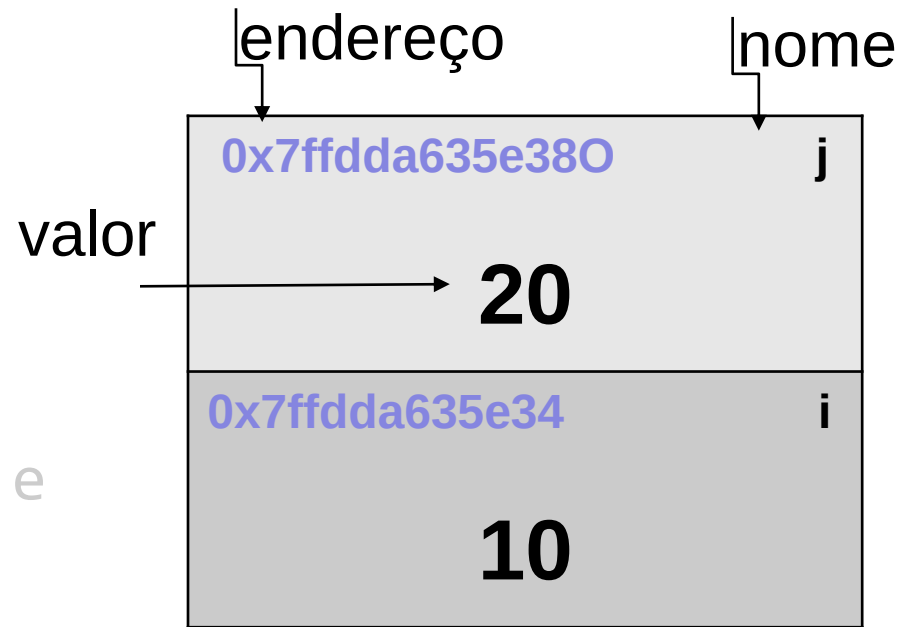
Alocação de variáveis

- Quando uma variável é alocada, é reservado um espaço da memória para armazenar o dado atribuído a variável.
- Esse espaço de memória possui um endereço, mas no programa, esse endereço é manipulado através do nome da variável

Representação abstrata da memória

```
#include <stdio.h>
```

```
int main(void) {  
    int i = 10, j = 20;  
    printf("O valor de i e  
    %d", i);  
    printf("\nO valor de j e  
    %d", j);  
    return 0;  
}
```



Alocação de variáveis

- Para saber o endereço de uma variável, basta colocar o & na frente do nome da variável

i é o valor de i

&i é o endereço onde i está alocada

Representação abstrata da memória

```
#include <stdio.h>
```

```
int main(void) {  
    int i = 10, j;  
    printf("O valor de i e %d  
e o endereço de i e %p",  
i, &i);  
    printf("\nO valor de j e  
%d e o endereço de i e  
%p", j, &j);  
    return 0;  
}
```

0x7ffdda635e380	j
20	
0x7ffdda635e34	i
10	

Ponteiro

- Em algumas situações, você pode precisar manipular o endereço de uma variável e não somente o valor dela, para isso, você deve usar os ponteiros
- Ponteiros são variáveis que armazenam o endereço de memória de outras variáveis
- Quando uma variável ponteiro “aponta” para outra variável *i*, a variável ponteiro guarda o endereço de *i*.

Ponteiro

- Como declarar uma variável que é um ponteiro?

- Usando operador * na da variável

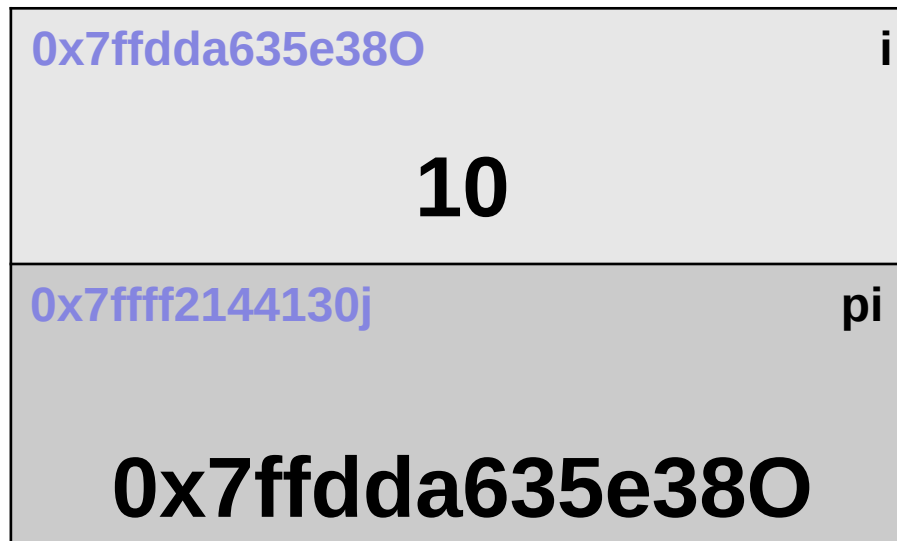
`int i;` // i é uma variável que guarda um valor int

`int *pi;` // pi é uma variável que guarda um
// endereço onde será armazenado um valor
// int

// pi só pode guardar um endereço de
// memória que armazena um valor int

Ponteiros

```
#include <stdio.h>
int main(void) {
    int i = 10;
    int *pi;
    pi = &i; // pi aponta para i
    printf("O valor de i e %d e  
o endereco de i e %p", i,  
&i);
    printf("\nO valor de pi e  
%p", pi);
    return 0;
}
```



p é uma variável
que aponta para
variável i

Ponteiro

- Como declarar uma variável que é um ponteiro?
 - Usando o mesmo operador da declaração o *

```
int i;
```

```
int *pi;
```

```
pi = &i;
```

```
*pi = 10; // o endereço apontado por pi vai  
          // receber o valor 10
```

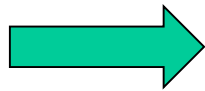
```
// como pi aponta para i, i recebe 10
```

Ponteiro

- Cuidado com ponteiros não inicializados

```
int i, *pi;
```

```
*pi = 10;
```



pi está apontando para qual endereço??

Todo ponteiro antes de ser usado deve ser inicializado corretamente, ou seja, deve apontar para um endereço físico válido para seu programa (endereços que foram alocados para o seu programa, no nosso caso, esses endereços são as variáveis alocadas no programa)